



ANÁLISIS Y DISEÑO DE ALGORITMOS

Resolución de Problemas mediante Programación Dinámica en C++

Curso:	Análisis y Diseño de Algoritmos
Tema:	Programación Dinámica
Estudiante:	Bran Yames Paz

Objetivo del Trabajo

Objetivo General:

Aplicar la estrategia de programación dinámica (PD) para resolver problemas de optimización, además de analizar el comportamiento y complejidad de los algoritmos implementados.

1. Escaleras

Un personaje de videojuego se encuentra frente a una escalera con N escalones y puede avanzar únicamente de 1 en 1 o de 2 en 2 escalones. Se desea calcular de cuántas formas diferentes puede llegar exactamente al escalón N .

Estrategia PD

Estado DP

Definimos un arreglo dp de tamaño $N + 1$ donde cada elemento $dp[i]$ representa el número total de formas diferentes de llegar exactamente al escalón i .

Relación de Recurrencia

Para cualquier escalón $i \geq 2$, la última acción para llegar a él debió ser dar un paso de 1 escalón (desde el escalón $i - 1$) o dar un paso de 2 escalones (desde el escalón $i - 2$). Por el principio de la suma, el número de formas de llegar al escalón i es la suma de las formas de llegar a esos dos escalones previos:

$$dp[i] = dp[i - 1] + dp[i - 2]$$

Casos Base

- **$dp[0] = 1$** : Existe exactamente una forma de estar en el suelo (escalón 0), que consiste en no realizar ningún movimiento.
- **$dp[1] = 1$** : Existe exactamente una forma de llegar al escalón 1, que consiste en dar un único paso de tamaño 1.

Análisis de Complejidad

Aspecto	Complejidad	Justificación
Temporal	$O(N)$	El algoritmo calcula linealmente los valores de la tabla DP desde 2 hasta N de forma secuencial.
Espacial	$O(N)$	Se requiere una tabla (vector) de tamaño $N + 1$ para almacenar los subproblemas.

Codigo

```
#include <iostream>
#include <vector>
using namespace std;

//Funcion para calcular las formas de subir N escalones
void escalera(int n) {
    //si n es negativo sale de la funcion
    if (n < 0) return;

    //crea un vector dp de tamaño n+1 para almacenar el número de formas de
    llegar a cada escalón
    vector<long long> dp(n + 1, 0);
```

```

    //dp[0] = 1 para que 1 este en el inicio y dp[1] = 1 para que 1 este en el
    primer escalon
    dp[0] = 1;
    if (n >= 1) {
        dp[1] = 1;
    }
    for (int i = 2; i <= n; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    cout << "Formas posibles: " << dp[n] << endl;

    cout << "Tabla dp: ";
    for (int i = 0; i <= n; ++i) {
        cout << dp[i];
        if (i < n) cout << " ";
    }
}

int main() {
    int n;
    cout << "Ingrese el numero de escalones N: ";
    cin >> n;
    escalera(n);
    return 0;
}

```

Ejemplos de Ejecución y Validación - Escaleras

Caso de Validación 1: N = 5

```

Ingrese el numero de escalones N: 5
Formas posibles: 8
Tabla dp: 1 1 2 3 5 8

```

Caso de Validación 2: N = 7

```

Ingrese el numero de escalones N: 7
Formas posibles: 21
Tabla dp: 1 1 2 3 5 8 13 21

```

Tiene casi el mismo comportamiento de fibonacci, el valor $dp[7]$ se calcula como $dp[6] + dp[5] = 13 + 8 = 21$ formas.

2. Cambio mínimo de monedas

Una máquina expendedora debe entregar una cantidad exacta de dinero utilizando la menor cantidad posible de monedas. Contamos con un conjunto de denominaciones de monedas y una cantidad objetivo. Buscamos determinar el número mínimo de monedas necesarias para formar dicha cantidad.

Definición de la Estrategia PD

Estado DP

Definimos un arreglo **dp** de tamaño Cantidad + 1 donde cada posición **dp[i]** representa el número mínimo de monedas necesarias para formar exactamente la cantidad de dinero *i*.

Relación de Recurrencia

Para formar la cantidad *i*, podemos tomar cualquier moneda disponible *c* (siempre que $c \leq i$) y sumarle 1 al número mínimo de monedas necesarias para formar la cantidad remanente $i - c$. Buscamos el mínimo sobre todas las opciones posibles de monedas:
$$dp[i] = \min_{c \in \text{Monedas}} \{ dp[i - c] + 1 \}$$

Casos Base

- **dp[0] = 0** : Se necesitan exactamente 0 monedas para formar una cantidad de 0.
- **dp[i] = actual** (inicialmente para $i > 0$): Representa que inicialmente no conocemos una combinación válida para formar la cantidad *i*.

Análisis de Complejidad

Aspecto	Complejidad	Justificación
Temporal	$O(N * M)$	Donde N es la cantidad objetivo y M es el número de denominaciones de monedas disponibles, para cada cantidad de 1 a N evaluamos la transición para cada una de las M monedas.
Espacial	$O(N)$	Se requiere una tabla de tamaño $N + 1$ para almacenar el número mínimo de monedas para todas las cantidades previas desde 0 hasta N.

Codigo

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// funcion para resolver el problema del cambio minimo de monedas
void CambioMoneda(const vector<int>& monedas, int cant_llegar) {
    const int actual = cant_llegar + 2;
    vector<int> dp(cant_llegar + 1, actual);
    //dp[0] = 0 0 monedas para hacer cantidad 0
    dp[0] = 0;

    // dp[i] representa el numero minimo de monedas necesarias para formar la
    cantidad i
    for (int i = 1; i <= cant_llegar; ++i) {
        for (int c : monedas) {
            if (i - c >= 0) {
                //relacion de recurrencia: dp[i] = min(dp[i], dp[i - c] + 1)
                para cada moneda c
                dp[i] = min(dp[i], dp[i - c] + 1);
            }
        }
    }

    //salir de la funcion si no es posible formar la cantidad deseada
    if (dp[cant_llegar] >= actual) return;

    cout << "Cantidad minima de monedas: " << dp[cant_llegar] << endl;

    // Reconstruccion de la combinacion de monedas utilizada
```

```

        vector<int> combinacion;
        int totalAhora = cant_llegar;
        while (totalAhora > 0) {
            // Iterar sobre las monedas para encontrar una que haya sido utilizada
            // en la solucion optima
            for (int c : monedas) {
                // Verificar si la moneda c fue utilizada en la solucion optima
                if (totalAhora - c >= 0 && dp[totalAhora] == dp[totalAhora - c] +
1) {
                    combinacion.push_back(c);
                    totalAhora -= c;
                    break;
                }
            }
        }

        // Ordenar la combinacion de mayor a menor
        sort(combinacion.rbegin(), combinacion.rend());

        cout << "Combinacion: ";
        for (size_t i = 0; i < combinacion.size(); ++i) {
            cout << combinacion[i];
            if (i < combinacion.size() - 1) {
                cout << " + ";
            }
        }

        // Imprimir la tabla de DP
        cout << "\nTabla DP:" << endl;
        for (int i = 0; i <= cant_llegar; ++i) {
            int val = (dp[i] >= actual) ? -1 : dp[i];
            cout << val;
            if (i < cant_llegar) cout << ", ";
        }
    }

    int main() {
        int cant;
        cout << "Ingrese la cantidad de denominaciones de monedas: ";
        if (!(cin >> cant)) return 0;

        // crear un vector para almacenar las denominaciones de monedas
        vector<int> monedas(cant);
        cout << "Ingrese las denominaciones de las monedas: ";
        for (int i = 0; i < cant; ++i) {
            cin >> monedas[i];
        }

        int cant_llegar;
        cout << "Ingrese la cantidad a la que se desea llegar: ";
        cin >> cant_llegar;
        CambioMoneda(monedas, cant_llegar);

        return 0;
    }
}

```

Ejemplos de Ejecución y Validación - Monedas

```
Ingrese la cantidad de denominaciones de monedas: 2
Ingrese las denominaciones de las monedas: 10
2
Ingrese la cantidad a la que se desea llegar: 12
Cantidad minima de monedas: 2
Combinacion: 10 + 2
Tabla DP:|
0, -1, 1, -1, 2, -1, 3, -1, 4, -1, 1, -1, 2
```

Caso de Validación 2: Monedas [1, 2, 5], Cantidad = 11

```
Ingrese la cantidad de denominaciones de monedas: 3
Ingrese las denominaciones de las monedas: 1
2
5
Ingrese la cantidad a la que se desea llegar: 11
Cantidad minima de monedas: 3
Combinacion: 5 + 5 + 1
Tabla DP:
0, 1, 1, 2, 2, 1, 2, 2, 3, 3, 2, 3
```

Explicación para cantidad 11: La solución óptima es usar dos monedas de 5 y una de 1 ($5 + 5 + 1 = 11$), dando un total de 3 monedas. El valor $dp[11]$ es 3. El algoritmo retrocede en la tabla DP identificando qué monedas mantuvieron la igualdad en la relación de recurrencia.